Competitive Programming Reference

All snippets except Modular Arithmetic template and Big Integer template

MOAZ KING

```
/**
 *    author:  MOAZ_KING
 *    created: $1
**/
```

tc

```
#include <bits/stdc++.h>
using namespace std;
int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);
  int tt;
  cin >> tt;
  while (tt--) {
    $1
  }
  return 0;
}
```

notc

```
#include <bits/stdc++.h>
using namespace std;
int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);
  $1
  return 0;
}
```

dxdy

```
int dx[] {0, 0, 1, -1, 1, -1, -1, 1};
int dy[] {1, -1, 0, 0, 1, -1, 1, -1};
```

primeFactorization

```
auto primeFact(int64_t n) {
  map<int64_t, int64_t> mp;
  for (int64_t i = 2; i * i <= n; i++) {
    while (!(n % i)) {
      n /= i, mp[i]++;
    }
  }
  if (n > 1) {
    mp[n]++;
  }
  return mp;
}
```

dSum

```
int64_t dSum(int64_t n) {
  int64_t sum = 0;
```

```
  while (n) {
    sum += n % 10;
    n /= 10;
  }
  return sum;
}
```

binaryToDecimal

```
int64_t bintodec(string s) {
  return stoll(s, 0, 2);
}
```

decimalToBinary

```
string dectobin(int64_t n) {
  if (!n) {
    return "0";
  }
  string s = bitset<64>(n).to_string();
  return s.substr(s.find_first_not_of('0'));
}
```

factorial

```
int64_t fact(int64_t n, int64_t s = 2) {
  int64_t res = 1;
  for (int64_t i = s; i <= n; i++) {
    res *= i;
  }
  return res;
}
```

fastPower

```
int64_t fp(int64_t b, int64_t p) {
  if (!p) return 1;
  int64_t hp = fp(b, p >> 1);
  int64_t f = hp * hp;
  return p & 1 ? f * b : f;
}
```

fastPowerMod

```
int64_t MOD = 1e9 + 7;
int64_t fpm(int64_t b, int64_t p) {
  if (!p) return 1;
  int64_t x = fpm(b, p >> 1);
  x = (x % MOD) * (x % MOD) % MOD;
  return p & 1 ? (x % MOD) * (b % MOD) % MOD : x;
}
```

fixMod

```
int64_t fixMod(int64_t a, int64_t b) {
  return (a % b + b) % b;
}
```

mpCompare

```
template<typename T1, typename T2>
bool mpcomp(pair<T1, T2> p1, pair<T1, T2> p2) {
  return p1.second < p2.second;
}
```

orderedMultiset

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
using namespace std;
template <typename T>
using ordered_multiset = tree<T, null_type, less_equal<T>, rb_tree_tag, tree_order_statistics_node_update>;
```

orderedSet

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
using namespace std;
template <typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;
```

primeChecker

```
bool isPrime(int64_t n) {
  if (n == 2) {
    return true;
  } else if (!(n % 2) || n == 1) {
    return false;
  }
  for (int64_t i = 3; i * i <= n; i += 2) {
    if (!(n % i)) {
      return false;
    }
  }
  return true;
}
```

rotateString

```
void operator<<=(string &s, int n) {
  string f = s + s;
  s = f.substr(n % s.length(), s.length());
}
void operator>>=(string &s, int n) {
  s <<= s.length() - (n % s.length());
}
```

sieve

```
bitset<100000> compo;
void sieve(int64_t n) {
  compo[0] = compo[1] = 1;
  for (int i = 2; i * i <= n; i++) {
    for (int j = i * i; j <= n; j += i) {
      compo[j] = 1;
    }
  }
}
```

smallestPrimeFactor

```
vector<short> spf($1 + 10);
for (int i = 2; i * i <= $1; i++) {
  if (spf[i]) {
    continue;
  }
  for (int j = i * i; j <= $1; j += i) {
    if (!spf[j]) {
      spf[j] = i;
    }
  }
}
```

combinatorics

```
int64_t nCr(int64_t n, int64_t r) {
  if (r > n || r < 0) {
    return 0;
  }
  if (r > n - r) {
    r = n - r;
  }
  int64_t res = 1;
  for (int i = 0; i < r; i++) {
    res *= (n - i);
    res /= (i + 1);
  }
  return res;
}
int64_t nPr(int64_t n, int64_t r) {
  if (r > n || r < 0) {
    return 0;
  }
  int64_t res = 1;
  for (int i = 0; i < r; i++) {
    res *= n - i;
  }
  return res;
}
```

leet

```
#include <bits/stdc++.h>
using namespace std;
class Solution {
public:
  $1
};
int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  Solution sol;
  return 0;
}
```

inout

```
freopen("$1.in", "r", stdin);
freopen("$1.out", "w", stdout);
```

randomNumber

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
int64_t rnd(int64_t l, int64_t r) {
  return uniform_int_distribution<int64_t>(l, r)(rng);
}
```

runtime

```
auto begin = std::chrono::high_resolution_clock::now();
auto end = std::chrono::high_resolution_clock::now();
auto elapsed = std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin);
cerr << "Time measured: " << elapsed.count() * 1e-9 << " seconds.\n";
```

assertion

```
void Assert(bool f, int code) {
  if (!f) {
    cerr << "Assertion: " << code << " failed.\n";
    exit(code);
  }
}
```

seg tree

```
struct Node {
  int64_t sum;
  Node() { sum = 0; }
  Node(int64_t x) { sum = x; }
  void operator=(const Node other) { sum = other.sum; }
};
template <typename T, typename U, auto merge>
struct SegmentTree {
  int n;
  vector<T> tree;
  inline int left(int node) { return node << 1; }
  inline int right(int node) { return node << 1 | 1; }
  void update(int idx, U val) { update(1, 0, n - 1, idx, val); }
  T query(int l, int r) { return query(1, 0, n - 1, l, r); }
  SegmentTree() { }
  SegmentTree(const vector<U> &v) { build(v); }
  void build(const vector<U> &v) {
    n = v.size();
    tree.resize(n << 2, T());
    build(1, 0, n - 1, v);
  }
  void build(int node, int l, int r, const vector<U> &v) {
    if (l == r) {
      tree[node] = T(v[l]);
      return;
    }
    int m = l + r >> 1;
    build(left(node), l, m, v);
    build(right(node), m + 1, r, v);
    tree[node] = merge(tree[left(node)], tree[right(node)]);
  }
  void update(int node, int l, int r, int idx, U &val) {
    if (l == r) {
      tree[node] = T(val);
      return;
    }
    int m = l + r >> 1;
    if (idx <= m) {
      update(left(node), l, m, idx, val);
    } else {
      update(right(node), m + 1, r, idx, val);
    }
    tree[node] = merge(tree[left(node)], tree[right(node)]);
  }
  T query(int node, int l, int r, int lx, int rx) {
    if (l >= lx && r <= rx) return tree[node];
    int m = l + r >> 1;
    T l_ans = lx <= m ? query(left(node), l, m, lx, rx) : T();
    T r_ans = rx > m ? query(right(node), m + 1, r, lx, rx) : T();
    return merge(l_ans, r_ans);
  }
};
inline Node merge_q(const Node &a, const Node &b) {
  // merge queries
}
```

spa table

```
struct Node {
  int mn;
  Node() { mn = 0; }
  Node(int x) { mn = x; }
  void operator=(const Node other) { mn = other.mn; }
};
template <typename T, typename U, auto merge>
struct SparseTable {
  int n;
  vector<vector<T>> table;
  SparseTable() { }
  SparseTable(const vector<U> &v) { build(v); }
  void build(const vector<U> &v) {
    n = v.size();
    table = vector<vector<T>>(n, vector<T>(__lg(n) + 1));
    for (int i = 0; i < n; i++) {
      table[i][0] = T(v[i]);
    }
    for (int p = 1; p <= __lg(n); p++) {
      for (int i = 0; i <= n - (1 << p); i++) {
        table[i][p] = merge(table[i][p - 1], table[i + (1 << (p - 1))][p - 1]);
      }
    }
  }
  T query(int l, int r) {
    int p = __lg(r - l + 1);
    return merge(table[l][p], table[r - (1 << p) + 1][p]);
  }
};
inline Node merge_q(const Node &a, const Node &b) {
  return Node(min(a.mn, b.mn));
}
```

lazy seg tree

```
struct TreeNode {
  int64_t sum;
  TreeNode() { sum = 0; }
  TreeNode(int64_t x) { sum = x; }
  void operator=(const TreeNode other) { sum = other.sum; }
};
struct LazyNode {
  int64_t sum;
  LazyNode() { sum = 0; }
  LazyNode(int64_t x) { sum = x; }
  void operator=(const LazyNode other) { sum = other.sum; }
  bool operator==(const LazyNode other) const { return sum == other.sum; }
};
template <typename T, typename U, typename V, auto merge_q, auto merge_u, auto apply>
struct SegmentTree {
  int n;
  vector<T> tree;
  vector<V> lazy;
  inline int left(int node) { return node << 1; }
  inline int right(int node) { return node << 1 | 1; }
  void update(int l, int r, V val) { update(1, 0, n - 1, l, r, val); }
  T query(int l, int r) { return query(1, 0, n - 1, l, r); }
  SegmentTree() { }
  SegmentTree(const vector<U> &v) { build(v); }
  void build(const vector<U> &v) {
    n = v.size();
    tree.resize(4 * n, T());
    lazy.resize(4 * n, V());
    build(1, 0, n - 1, v);
  }
  void push(int node, int l, int r) {
    if (lazy[node] == V()) return;
    if (l < r) {
      lazy[left(node)] = merge_u(lazy[node], lazy[left(node)]);
      lazy[right(node)] = merge_u(lazy[node], lazy[right(node)]);
    }
    apply(tree[node], lazy[node], l, r);
    lazy[node] = V();
```

```
  }
  void build(int node, int l, int r, const vector<U> &v) {
    if (l == r) {
      tree[node] = T(v[l]);
      return;
    }
    int m = l + r >> 1;
    build(left(node), l, m, v);
    build(right(node), m + 1, r, v);
    tree[node] = merge_q(tree[left(node)], tree[right(node)]);
  }
  void update(int node, int l, int r, int lx, int rx, V &val) {
    push(node, l, r);
    if (l > rx || r < lx) return;
    if (l >= lx && r <= rx) {
      lazy[node] = merge_u(val, lazy[node]);
      push(node, l, r);
      return;
    }
    int m = l + r >> 1;
    update(left(node), l, m, lx, rx, val);
    update(right(node), m + 1, r, lx, rx, val);
    tree[node] = merge_q(tree[left(node)], tree[right(node)]);
  }
  T query(int node, int l, int r, int lx, int rx) {
    push(node, l, r);
    if (r < lx || l > rx) return T();
    if (l >= lx && r <= rx) return tree[node];
    int m = l + r >> 1;
    T l_ans = query(left(node), l, m, lx, rx);
    T r_ans = query(right(node), m + 1, r, lx, rx);
    return merge_q(l_ans, r_ans);
  }
};
inline TreeNode merge_q(const TreeNode &a, const TreeNode &b) {
  // merge queries.
}
inline LazyNode merge_u(const LazyNode &a, const LazyNode &b) {
  // a is the new lazy.
  // b is the already existed lazy.
  // merge updates.
}
inline void apply_u(TreeNode &a, LazyNode &b, int l, int r) {
  // apply lazy update to a tree node.
}
```

DSU

```
struct DSU {
  int n;
  vector<int> par, sz;
  DSU(int x) : n(x) {
    par = vector<int>(n + 1);
    sz = vector<int>(n + 1, 1);
    iota(par.begin(), par.end(), 0);
  }
  int find(int u) {
    if (par[u] == u) return u;
    return par[u] = find(par[u]);
  }
  void unite(int u, int v) {
    int p1 = find(u);
    int p2 = find(v);
    if (p1 == p2) return;
    if (sz[p1] > sz[p2]) swap(p1, p2);
    par[p1] = p2;
    sz[p2] += sz[p1];
  }
  bool same(int u, int v) { return find(u) == find(v); }
  int size(int u) { return sz[find(u)]; }
};
```

fast scanner

```
struct FastScanner {
```

```
  static constexpr int BUFSIZE = 1 << 16;
  int idx = 0, size = 0;
  char buf[BUFSIZE];
  inline char readChar() {
    if (idx >= size) {
      size = (int)fread(buf, 1, BUFSIZE, stdin);
      idx = 0;
      if (size == 0) return 0;
    }
    return buf[idx++];
  }
  template <class T>
  bool nextInt(T& out) {
    char c = readChar();
    if (!c) return false;
    while (c <= ' ') {
      c = readChar();
      if (!c) return false;
    }
    bool neg = false;
    if (c == '-') {
      neg = true;
      c = readChar();
    }
    int64_t val = 0;
    while (c > ' ') {
      val = val * 10 + (c - '0');
      c = readChar();
    }
    out = neg ? (T)-val : (T)val;
    return true;
  }
  bool nextString(string& out) {
    out.clear();
    char c = readChar();
    if (!c) return false;
    while (c <= ' ') {
      c = readChar();
      if (!c) return false;
    }
    while (c > ' ') {
      out.push_back(c);
      c = readChar();
    }
    return true;
  }
};
```

MO's Algorithm

```
struct Query {
  int l, r, ind;
  Query() { l = r = -1; }
  Query(int a, int b, int c) : l(a), r(b), ind(c) { }
  bool operator<(const Query &other) const {
    if (l / SQ != other.l / SQ) {
      return l / SQ < other.l / SQ;
    }
    return (l / SQ) & 1 ? r > other.r : r < other.r;
  }
};
vector<int> MO(vector<Query> &que) {
  int q = que.size();
  vector<int> ans(q);
  sort(que.begin(), que.end());
  auto add = [&] (int i) {

  };
  auto rem = [&] (int i) {

  };
  int l = 0, r = -1;
  for (auto [L, R, ind] : que) {
    while (r < R) add(++r);
    while (l > L) add(--l);
    while (r > R) rem(r--);
    while (l < L) rem(l++);
    ans[ind] = ;
```

```
  }
  return ans;
}
```

LCA

```
struct LCA {
  int n, lg;
  vector<vector<int>> up;
  vector<int> dep;
  LCA() { }
  void dfs(vector<vector<int>> &adj, int node, int u) {
    for (auto &child : adj[node]) {
      if (child == u) continue;
      dep[child] = 1 + dep[node];
      up[child][0] = node;
      for (int i = 1; i <= lg; i++) {
        up[child][i] = up[up[child][i - 1]][i - 1];
      }
      dfs(adj, child, node);
    }
  }
  LCA(vector<vector<int>> &adj, int rt) {
    n = (int)adj.size();
    lg = __lg(n) + 1;
    dep = vector<int>(n + 1);
    up = vector<vector<int>>(n + 1, vector<int>(lg + 1));
    dfs(adj, rt, -1);
  }
  int kth_anc(int u, int k) {
    if (k > dep[u]) return -1;
    for (int i = lg; i >= 0; i--) {
      if (k >> i & 1) {
        u = up[u][i];
      }
    }
    return u;
  }
  int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    int k = dep[u] - dep[v];
    u = kth_anc(u, k);
    if (u == v) return u;
    for (int i = lg; i >= 0; i--) {
      if (up[u][i] != up[v][i]) {
        u = up[u][i];
        v = up[v][i];
      }
    }
    return up[u][0];
  }
};
```

trie

```
struct Node {
  int cnt {};
};
struct Trie {
  int n {};
  vector<array<int, 26>> adj;
  vector<Node> a;
  Trie() { adj.emplace_back(), a.emplace_back(); }
  void insert(const string &s) {
    int node = 0;
    for (int i = 0; i < s.length(); i++) {
      if (adj[node][s[i] - 'a']) {
        node = adj[node][s[i] - 'a'];
        a[node]++;
      } else {
        n++;
        a.emplace_back();
        adj.emplace_back();
        adj[node][s[i] - 'a'] = n;
        node = n;
```

```
      a[node]++;
    }
  }
}
int count(const string &s) {
  int node = 0;
  for (int i = 0; i < s.length(); i++) {
    if (adj[node][s[i] - 'a']) {
      node = adj[node][s[i] - 'a'];
    } else {
      return 0;
    }
  }
  return a[node].cnt;
}
};
```

double hashing

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
int64_t rnd(int64_t l, int64_t r) { return uniform_int_distribution<int64_t>(l, r)(rng); }
const int64_t MOD1 = 1e9 + 7;
const int64_t MOD2 = 1e9 + 9;
const int64_t base1 = rnd(0.25 * MOD1, 0.75 * MOD1);
const int64_t base2 = rnd(0.25 * MOD1, 0.75 * MOD1);
struct HashedString {
  int64_t n {};
  vector<int64_t> hash1, hash2, pw1, pw2;
  HashedString() {
    hash1.emplace_back();
    hash2.emplace_back();
    pw1.push_back(1);
    pw2.push_back(1);
  }
  HashedString(const string &s) {
    hash1.emplace_back();
    hash2.emplace_back();
    pw1.push_back(1);
    pw2.push_back(1);
    build(s);
  }
  void build(const string &s) {
    for (int i = 0; i < int(s.length()); i++) {
      push_back(s[i]);
    }
  }
  void push_back(char c) {
    hash1.push_back((hash1.back() * base1 + c) % MOD1);
    pw1.push_back(pw1.back() * base1 % MOD1);
    hash2.push_back((hash2.back() * base2 + c) % MOD2);
    pw2.push_back(pw2.back() * base2 % MOD2);
    ++n;
  }
  pair<int64_t, int64_t> get(int l, int r) {
    auto h1 = (hash1[r + 1] - (hash1[l] * pw1[r - l + 1] % MOD1) + MOD1) % MOD1;
    auto h2 = (hash2[r + 1] - (hash2[l] * pw2[r - l + 1] % MOD2) + MOD2) % MOD2;
    return make_pair(h1, h2);
  }
};
```

fenwick tree

```
struct BIT {
  vector<int> tree;
  BIT(int n) { tree.assign(n + 5, {}); }
  int query(int r) {
    int res = 0;
    for (++r; r > 0; r -= r & -r) {
      res += tree[r];
    }
    return res;
  }
  int query(int l, int r) {
    return query(r) - query(l - 1);
  }
```

```
  void update(int i, int v) {
    for (++i; i < int(tree.size()); i += i & -i) {
      tree[i] += v;
    }
  }
};
```

kmp build prefix

```
template <typename T>
vector<int> kmp(const T &a) {
  int n = int(a.size());
  vector<int> pi(n);
  for (int i = 1; i < n; i++) {
    int j = pi[i - 1];
    while (j > 0 && a[i] != a[j]) {
      j = pi[j - 1];
    }
    if (a[i] == a[j]) j++;
    pi[i] = j;
  }
  return pi;
}
```

single hashing

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
int64_t rnd(int64_t l, int64_t r) { return uniform_int_distribution<int64_t>(l, r)(rng); }
const uint64_t base = rnd(1e6, 1e9);
struct HashedString {
  int64_t n {};
  vector<uint64_t> hsh, pw;
  HashedString() {
    hsh.emplace_back();
    pw.push_back(1);
  }
  HashedString(const string &s) {
    hsh.emplace_back();
    pw.push_back(1);
    build(s);
  }
  void build(const string &s) {
    for (int i = 0; i < int(s.length()); i++) {
      push_back(s[i]);
    }
  }
  void push_back(char c) {
    hsh.push_back(hsh.back() * base + c);
    pw.push_back(pw.back() * base);
    ++n;
  }
  uint64_t get(int l, int r) {
    return hsh[r + 1] - hsh[l] * pw[r - l + 1];
  }
};
```

Z function

```
template <typename T>
vector<int> Z_function(const T &a) {
  int n = int(a.size());
  vector<int> Z(n);
  int l = 0, r = 0;
  for (int i = 1; i < n; i++) {
    if (i <= r) {
      Z[i] = min(r - i, Z[i - l]);
    }
    while (i + Z[i] < n && a[Z[i]] == a[i + Z[i]]) {
      Z[i]++;
    }
    if (i + Z[i] > r) {
      l = i;
```

```
      r = i + Z[i];
    }
  }
  return Z;
}
```

manacher

```
template <typename T>
vector<int> manacher_odd(T a) {
  int n = int(a.size());
  a.insert(a.begin(), 0);
  a.push_back(1);
  vector<int> p(n + 2);
  int l = 0, r = 1;
  for (int i = 1; i <= n; i++) {
    if (i <= r) {
      p[i] = min(r - i, p[l + (r - i)]);
    }
    while (a[i - p[i]] == a[i + p[i]]) {
      p[i]++;
    }
    if (i + p[i] > r) {
      l = i - p[i];
      r = i + p[i];
    }
  }
  for (int i = 1; i <= n; i++) {
    p[i] = (p[i] << 1) - 1;
  }
  return vector<int>(p.begin() + 1, p.end() - 1);
}
template <typename T>
vector<int> manacher(const T &a) {
  int n = int(a.size());
  vector<int> all;
  for (int i = 0; i < n; i++) {
    all.push_back(-1);
    all.push_back(a[i]);
  }
  all.push_back(-1);
  auto m(manacher_odd(all));
  for (int i = 0; i < int(m.size()); i++) {
    m[i] >>= 1;
  }
  return m;
}
```

splitmix

```
struct SplitMix {
  static uint64_t splitmix64(uint64_t x) {
    x += 0x9e3779b97f4a7c15;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
    x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
    return x ^ (x >> 31);
  }
  uint64_t operator()(uint64_t x) const {
    static const uint64_t FIXED_RANDOM = chrono::steady_clock::now().time_since_epoch().count();
    return splitmix64(x + FIXED_RANDOM);
  }
};
```